

Algorithm Testing & Optimization

Author: Yasithzz | 2026

02 -- Algorithm Testing & Optimization

LifeMate Project -- English Documentation

1. What does app.py do?

ml-service/app.py exposes the trained model (produced by train.py) as a live HTTP API using the FastAPI framework. Think of the trained model as a warehouse of knowledge; app.py is the storefront -- it accepts requests from the Spring Boot backend, queries the model, and returns predictions. The service also supports live data ingestion and automatic retraining so the model improves as users submit confirmed data.

2. app.py -- Line-by-Line Explanation

2.1 -- Docstring

```
"""
Endpoints:
POST /predict -- predict stress level from lifestyle data
POST /add-data -- append a labelled row to the dataset (live learning)
POST /retrain -- retrain model from updated dataset
GET /health -- liveness probe
GET /model-info -- returns current model accuracy + feature list
"""
```

Five endpoints form the public surface of the ML service.

2.2 -- Imports

Import	Purpose
`threading`	Thread lock to prevent race conditions between predict and retrain
`joblib`	Loads the `.pkl` model binary into memory
`fastapi`	HTTP API framework
`CORSMiddleware`	Allows the React frontend to call the ML API cross-origin
`pydantic`	Validates and coerces request payloads

2.3 -- Path Constants

```
MODEL_PKL = os.path.join(BASE, "model", "stress_model.pkl")
MODEL_META= os.path.join(BASE, "model", "model_meta.json")
DATASET   = os.path.join(BASE, "..", "datasets", "stress_dataset.csv")
```

File locations for the serialised model, metadata JSON, and the live dataset used for retraining.

2.4 -- RETRAIN_BATCH and Thread Lock

```
RETRAIN_BATCH = 1
_new_samples_count = 0
_model_lock = threading.Lock()
```

* RETRAIN_BATCH = 1 -- after every single new confirmed data point, an automatic retrain is triggered.

* _model_lock -- a threading lock ensures that a prediction request and a concurrent retrain cannot corrupt the model reference simultaneously.

2.5 -- FastAPI App Setup

```
app = FastAPI(title="LifeMate ML Service", version="1.0.0")
app.add_middleware(CORSMiddleware, allow_origins=["*"], ...)
```

Creates the API server. CORS middleware is required so the React browser app can POST to the ML service without browser security rejections.

2.6 -- Model Loading (with Caching)

```
_clf = None

def get_clf():
    global _clf
    if _clf is None:
        _clf = load_model()
    return _clf
```

The model is loaded from disk once and cached in _clf. Subsequent requests reuse the in-memory object -- no repeated disk I/O per prediction.

2.7 -- LifestyleInput Schema

```

class LifestyleInput(BaseModel):
    mood: int = Field(..., ge=1, le=5)
    workload: int = Field(..., ge=1, le=5)
    sleep_hours: float = Field(..., ge=0, le=12)
    energy_level: int = Field(..., ge=1, le=5)
    social_interaction: int = Field(3, ge=1, le=5)
    exercise_done: int = Field(0, ge=0, le=1)
    screen_time_hours: float = Field(4.0, ge=0, le=14)
    water_cups: float = Field(6.0, ge=0, le=16)

```

Pydantic automatically validates every incoming request. If any value is out of range, a 422 Unprocessable Entity error is returned before the model is ever called. Fields with defaults (social_interaction, exercise_done, etc.) are optional in the request body.

2.8 -- /health Endpoint

```

@app.get("/health")
def health():
    return {"status": "ok", "model_loaded": os.path.exists(MODEL_PKL)}

```

Used by infrastructure health checks and the Spring Boot backend to verify the ML service is alive and the model file exists.

2.9 -- /model-info Endpoint

```

@app.get("/model-info")
def model_info():
    with open(MODEL_META) as f:
        return json.load(f)

```

Returns the contents of model_meta.json: feature names, class labels, and the last recorded accuracy. Useful for admin dashboards.

2.10 -- /predict Endpoint (Core Endpoint)

```

@app.post("/predict", response_model=PredictResponse)
def predict(data: LifestyleInput):
    clf = get_clf()
    X = np.array([[
        data.mood, data.workload, data.sleep_hours, data.energy_level,
        data.social_interaction, data.exercise_done,
        data.screen_time_hours, data.water_cups,
    ]])
    with _model_lock:
        pred = int(clf.predict(X)[0])
        proba = clf.predict_proba(X)[0]

    return PredictResponse(
        stress_level=pred,
        stress_label=LABELS[pred],
        probabilities={LABELS[i]: round(float(p), 4) for i, p in enumerate(proba)},
    )

```

Step-by-step:

- * Pydantic validates the 8 input values.
- * `get_clf()` returns the in-memory model.
- * Inputs are reshaped into a 1×8 NumPy matrix -- the format scikit-learn expects.
- * `clf.predict(X)` -- the forest of 200 trees votes; the majority class index is returned.
- * `clf.predict_proba(X)` -- each tree's probability distribution is averaged.
- * Response JSON includes the numeric class, the human label, and the full probability breakdown.

```

{
  "stress_level": 2,
  "stress_label": "Normal",
  "probabilities": {
    "Very Low": 0.05,
    "Low": 0.10,
    "Normal": 0.65,
    "High": 0.15,
    "Very High": 0.05
  }
}

```

2.11 -- /add-data Endpoint (Live Learning)

```
@app.post("/add-data")
def add_data(data: AddDataRequest, background_tasks: BackgroundTasks):
    row = [data.mood, ..., data.stress_level]
    with open(DATASET, "a", newline="") as f:
        csv.writer(f).writerow(row)
    _new_samples_count += 1
    if _new_samples_count >= RETRAIN_BATCH:
        _new_samples_count = 0
        background_tasks.add_task(_do_retrain)
```

Appends a confirmed data point (with its verified stress label) to the CSV dataset. Because RETRAIN_BATCH = 1, a background retrain is triggered immediately after each new row.

2.12 -- /retrain Endpoint

```
@app.post("/retrain")
def retrain_endpoint(background_tasks: BackgroundTasks):
    background_tasks.add_task(_do_retrain)
    return {"status": "retraining started in background"}
```

Allows the backend or an admin to manually trigger a retrain without waiting for new data.

2.13 -- _do_retrain() Background Task

```
def _do_retrain():
    global _clf
    result = subprocess.run([sys.executable, os.path.join(BASE, "train.py")],
                           capture_output=True, text=True, cwd=BASE)
    if result.returncode == 0:
        with _model_lock:
            _clf = load_model()
```

- * Runs python train.py as a subprocess.
 - * On success, acquires the thread lock and replaces _clf with the newly trained model.
 - * The API continues serving predictions on the old model during retraining -- zero downtime.
-

3. Complete Prediction Flow

```
User (React Frontend)
  v POST /predict -- 8 lifestyle values (JSON)
FastAPI app.py
  v Pydantic validation
  v np.array -- reshape to 1x8 matrix
  v RandomForestClassifier.predict() -- majority vote across 200 trees
  v predict_proba() -- probability per class
  v JSON response
Spring Boot Backend
  v stress_level, stress_label, probabilities received
  v Saved to Lifestyle record in MongoDB
  v Weekly schedule generated (stress-aware)
Frontend
  v Displays stress result card + refreshes schedule page
```

Documentation -- LifeMate Project, Yasithzz, 2026